SOP Git Branching Strategy & Workflow

Status: Professional Standard Operating Procedure

Target Audience: All Programmers, DevOps, Tech Leads, QA

Objective: Menetapkan standar arsitektur cabang (branch) pada Git untuk menjaga stabilitas kode produksi, meminimalisir *merge conflict*, dan memastikan kompatibilitas penuh dengan sistem CI/CD otomatis.

1. TOPOLOGI CABANG (BRANCH TYPES)

Sistem repository kita menggunakan 4 tipe *branch* utama. Seluruh tim wajib mematuhi peruntukan masing-masing cabang ini.

🌿 main / master (Production Branch)

Cabang sakral yang berisi kode 100% stabil dan siap rilis ke pemain/klien.

- **Status:** Protected Branch (Dilindungi).
- **Aturan:** * DILARANG KERAS melakukan *direct commit* atau *push* langsung ke cabang ini.
    - Hanya menerima *Pull Request* (PR) yang sudah di-review dari `staging` atau `hotfix`.
- **CI/CD Trigger:** Pembuatan "Release Tag" dari cabang ini akan memicu *Auto Production Build*.

🧪 staging / develop (Integration Branch)

Pusat integrasi harian. Tempat berkumpulnya semua fitur yang dikerjakan oleh berbagai programmer.

- **Status:** Default Working Branch.
- **Aturan:**
    - Cabang utama untuk QA Tester melakukan uji coba (*playtest*).
    - Dilarang mengerjakan fitur berhari-hari langsung di cabang ini.
- **CI/CD Trigger:** Target utama untuk di-build menggunakan tombol *Manual Staging Build* di GitHub Actions.

🛠️ feature/* (Working Branch)

Cabang tempat programmer bekerja membuat fitur baru sehari-hari.

- **Format Nama:** `feature/[nama-fitur-atau-task-id]` (Contoh: `feature/inventory-system`, `feature/UI-revamp`).
- **Aturan:**
  - WAJIB dibuat bercabang (*branch out*) dari versi terbaru `staging`.
  - Jika pengerjaan memakan waktu lama, programmer wajib rutin melakukan `git pull origin staging` ke dalam branch ini agar tidak tertinggal jauh dan mencegah *conflict* besar di akhir.
  - Setelah selesai, buat *Pull Request* (PR) untuk digabungkan kembali ke `staging`.

🚑 hotfix/* (Emergency Branch)

Cabang darurat untuk memperbaiki *bug* kritis di versi produksi (game yang sudah rilis).

- **Format Nama:** `hotfix/[nama-bug-kritis]` (Contoh: `hotfix/login-crash`).
- **Aturan:**
  - WAJIB dibuat bercabang langsung dari `main`.
  - PENTING: Setelah perbaikan selesai, PR wajib di-merge ke `main` **DAN** di-merge juga ke `staging` agar *bug* yang sama tidak muncul lagi di pembaruan bulan depan.

2. WORKFLOW SIKLUS PENGEMBANGAN FITUR

Berikut adalah langkah-langkah standar saat seorang programmer ditugaskan membuat fitur baru:

1. **Sinkronisasi Awal:** Pindah ke branch staging dan ambil pembaruan terbaru.

   `git checkout staging` → `git pull origin staging`

2. **Buat Branch Baru:**

   Buat branch fitur dari staging.

   `git checkout -b feature/nama-fitur`

3. **Pengerjaan & Commit:**

   Kerjakan tugas, lalu commit dengan pesan yang jelas.

   `git commit -m "feat: Menambahkan sistem inventory dasar"`

4. **Push & Buka PR:**

   Push ke GitHub, lalu buka *Pull Request* (PR) menuju `staging`.

   `git push origin feature/nama-fitur`

5. **Review & Merge:**

Tech Lead / Peer akan mereview kode. Jika aman, kode di-merge ke `staging`. Branch fitur kemudian boleh dihapus.

6. **Testing (CI/CD):**

   Gunakan tombol *Manual Build Staging* untuk membuat versi *build* yang bisa dicoba oleh QA.

3. WORKFLOW PERILISAN (RELEASE KE MAIN)

Ketika `staging` dirasa sudah memiliki cukup fitur dan stabil, saatnya melakukan rilis:

1. Buka *Pull Request* (PR) dari `staging` menuju `main`.
2. Pastikan semua *automated test* (jika ada) berstatus *Passed*.
3. Lakukan Merge PR.
4. Buat Release baru di GitHub dari cabang `main` dan beri tag versi (contoh: `v1.2.0-beta`).
5. Sistem CI/CD akan otomatis membuat *Production Build*.